



Technical Documentation

dm-Verity Metadata Parsing (metadata_parse.c)

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

**BMW
GROUP**



ROLLS-ROYCE
MOTOR CARS LTD

[BMW Car IT, Ulm](#)

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

File Location

linux/drivers/md/dm-verity-autoboot/metadata_parse.c

1. Introduction

The metadata_parse.c module is responsible for extracting, converting, and logging the key parameters found within a verified dm-verity metadata header. Its primary function is to prepare these parameters for use by the dm-verity mapping module and to provide essential debugging output to the kernel log. This module operates on the assumption that the provided header has already passed cryptographic signature verification.

2. Binary-to-ASCII Encoding Helper

This module re-implements the hex_encode() function (also found in mapping.c) to convert binary hash and salt values into ASCII hexadecimal strings suitable for logging.

hex_encode() Function

- Purpose: Encodes a raw byte array into a null-terminated, lowercase hexadecimal ASCII string. This ensures that non-printable binary data can be safely viewed in the kernel log.
- Mechanism: Uses a simple static lookup array ("0123456789abcdef") and bitwise operations to convert each byte into two ASCII characters.

The code snippet is as follows:

```
33 static void hex_encode(const u8 *src, size_t len, char *dst)
34 {
35     static const char hexdig[] = "0123456789abcdef";
36     size_t i;
37
38     for (i = 0; i < len; i++) {
39         dst[2 * i] = hexdig[(src[i] >> 4) & 0xf];
40         dst[2 * i + 1] = hexdig[src[i] & 0xf];
41     }
42     dst[2 * len] = '\0';
43 }
```

3. Metadata Header Parsing and Logging

The core logic resides in the verity_parse_metadata_header() function. This function reads all critical fields, performs necessary byte and sector conversions, and outputs the detailed configuration to the kernel log using pr_info().

3.1. Data Extraction and Conversion

The function handles the conversion of fields from little-endian (on-disk format) to the CPU's native byte order using the leX_to_cpu macros. It also calculates derived metrics crucial for understanding the filesystem layout:

- Covered Bytes: The total size of the filesystem data protected by dm-verity, calculated as: `data_blocks * data_block_size`.
- Hash Start Bytes: The physical byte offset where the hash tree begins on the disk, calculated as: `hash_start_sector * 512ULL` (since 512 is the standard sector size).

The relevant data conversion code is:

```
65 | data_blocks      = le64_to_cpu(h->data_blocks);
66 | hash_start_sector = le64_to_cpu(h->hash_start_sector);
67 | data_block_bytes = (u64)le32_to_cpu(h->data_block_size);
68 | covered_bytes   = data_blocks * data_block_bytes;
69 | hash_start_bytes = hash_start_sector * 512ULL;
70 | salt_size       = le32_to_cpu(h->salt_size);
```

3.2. String Preparation and Safety

The hash algorithm name is copied and null-terminated. The `root_hash` and `salt` are converted to hex strings using the `hex_encode()` helper. A safety check limits the `salt_size` used for encoding to a maximum of 64 bytes, although the verity structure typically holds only 32 bytes for the hash and 64 for the salt. This bound check prevents buffer overruns in the `salt_hex` buffer:

```
75 | hex_encode(h->root_hash, 32, root_hash_hex);
76 |
77 | if (salt_size > 64)
78 |     salt_size = 64;
79 | hex_encode(h->salt, salt_size, salt_hex);
```

3.3. Debugging Output

The final part of the function logs all extracted and calculated values. This output is critical during debugging, providing confirmation that the dm-verity metadata header, which was verified by the `signature_verify` module, has been correctly interpreted by the kernel. The output includes key verification parameters (version, root hash, salt, and algorithm) and layout parameters (block sizes and offsets).

End of Document